2 Сортировка выбором



В этой главе

- ✓ Вы познакомитесь с массивами и связанными списками — двумя основными структурами данных, которые используются буквально везде. Мы уже применяли массивы в главе 1 и будем говорить о них почти в каждой главе книги. Массивы чрезвычайно важны, уделите им внимание! Впрочем, иногда вместо массива лучше воспользоваться связанным списком. В этой главе объясняются плюсы и минусы обеих структур данных, чтобы вы могли решить, какой вариант лучше подходит для вашего алгоритма.
- ✓ Вы изучите свой первый алгоритм сортировки. Многие алгоритмы работают только с отсортированными данными. Помните бинарный поиск? Он применяется только к предварительно отсортированному списку. В большинстве языков существуют встроенные алгоритмы сортировки, так что вам редко приходится писать свою версию «с нуля». Однако алгоритм сортировки выбором поможет перейти к алгоритму быстрой сортировки, описанному в главе 4. Алгоритм быстрой сортировки очень важен, и вам будет проще разобраться в нем, если вы уже знаете хотя бы один алгоритм сортировки.

ЧТО НЕОБХОДИМО ЗНАТЬ

Чтобы понять ту часть этой главы, которая относится к анализу эффективности, необходимо понимать смысл понятия «О-большое» и логарифмов. Если вы совершенно не разбираетесь в этих вопросах, лучше вернуться и прочитать главу 1. «О-большое» будет использоваться в оставшихся главах книги.

Как работает память

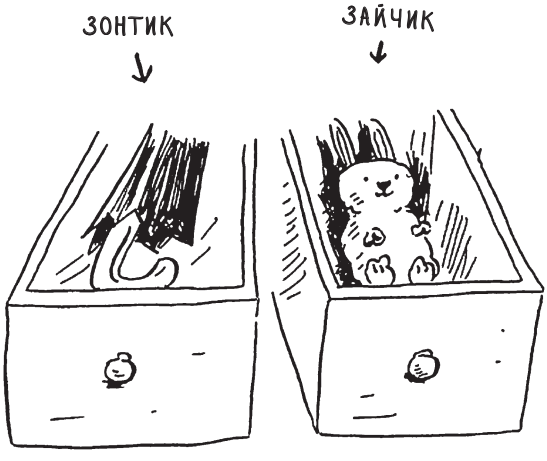
Представьте, что вы пришли в театр и хотите оставить свои личные вещи в гардеробе. Для хранения вещей есть специальные ящики.



В каждом ящике помещается один предмет. Вы хотите сдать на хранение две вещи, поэтому требуете выделить вам два ящика.

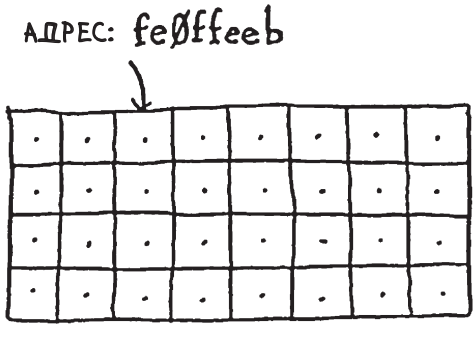


И вы оставляете свои две вещи.



Готово, можно идти на спектакль!

В сущности, именно так работает память вашего компьютера. Она представляет собой нечто вроде огромного шкафа со множеством ящиков, и у каждого ящика есть адрес.

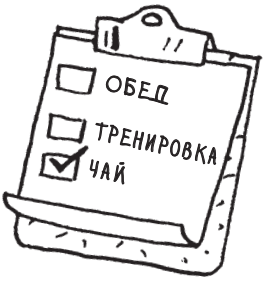


fe0ffeeb — адрес ячейки памяти.

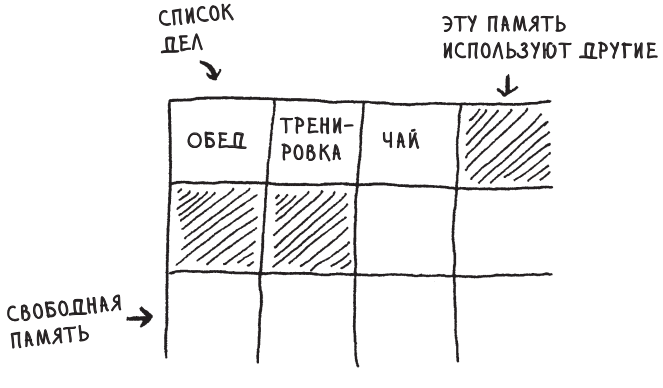
Каждый раз, когда вы хотите сохранить в памяти отдельное значение, вы запрашиваете у компьютера место в памяти, а он выдает адрес для сохранения значения. Если же вам понадобится сохранить несколько элементов, это можно сделать двумя основными способами: воспользоваться массивом или списком. В следующем разделе мы обсудим массивы и списки, их достоинства и недостатки. Не существует единственно верного способа сохранения данных на все случаи жизни, поэтому вы должны знать, в чем отличие разных способов.

Массивы и связанные списки

Иногда в памяти требуется сохранить список элементов. Предположим, вы пишете приложение для управления текущими делами. Описания задач должны храниться в виде списка в памяти.



Что использовать — массив или связанный список? Для начала попробуем сохранить задачи в массиве, потому что этот способ более понятен. При использовании массива все задачи хранятся в памяти непрерывно (то есть рядом друг с другом).



Теперь предположим, что вы захотели добавить четвертую задачу. Но следующий ящик уже занят — там лежат чужие вещи!



Представьте, что вы пошли в кино с друзьями и нашли места для своей компании, но тут приходит еще один друг, и ему сесть уже некуда. Приходится искать новое место, где смогут разместиться все. В нашем случае вам придется запросить у компьютера другой блок памяти, в котором поместятся все четыре задачи, а потом переместить все свои задачи туда.

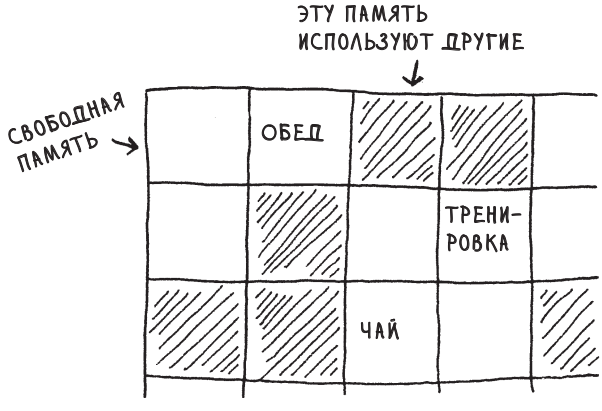
Если вдруг придет еще один друг, места опять не хватит и вам всем придется перемещаться снова! Сплошная суета. Кроме того, добавление новых элементов в массив станет серьезной проблемой. Если свободного места нет и вам каждый раз приходится перемещаться в новую область в памяти, операция добавления нового элемента будет выполняться очень медленно. Простейшее решение — «бронирование мест»: даже если список состоит всего из 3 задач, вы запрашиваете у компьютера место на 10 позиций... просто на всякий случай. Тогда в список можно будет добавить до 10 задач и ничего перемещать не придется. Это неплохое обходное решение, но у него есть пара недостатков.

- Лишнее место может не понадобиться, и тогда память будет расходоваться неэффективно. Вы ее не используете, однако никто другой ее использовать тоже не может.
- Если в список будет добавлено более 10 задач, перемещать все равно придется.

В общем, прием неплохой, но его нельзя назвать идеальным. Связанные списки решают проблему добавления новых элементов.

Связанные списки

При использовании связанного списка элементы могут размещаться где угодно в памяти.



В каждом элементе хранится адрес следующего элемента списка. Таким образом, набор произвольных адресов памяти объединяется в цепочку.



Связанные адреса памяти

Все как в игре «Найди клад». Вы приходите по первому адресу, там написано: «Следующий элемент находится по адресу 123». Вы идете по адресу 123, там написано: «Следующий элемент находится по адресу 847» и т. д. Добавить новый элемент в связанный список проще простого: просто разместите его по любому адресу памяти и сохраните этот адрес в предыдущем элементе.

Со связанными списками ничего перемещать в памяти не нужно. Также сама собой решается другая проблема: допустим, вы пришли в кино с пятью друзьями. Вы пытаетесь найти место на шестерых, но кинотеатр уже забит, и найти шесть соседних мест невозможно. Нечто похожее происходит и с массивами. Допустим, вы пытаетесь найти для массива блок на 10 000 элементов. В памяти можно найти место для 10 000 элементов, но только не смежное. Для массива не хватает места! При хранении данных в связанном списке вы фактически говорите: «Ладно, тогда садимся на свободные места и смотрим кино». Если необходимое место есть в памяти, вы сможете сохранить данные в связанном списке.

Если связанные списки так хорошо справляются со вставкой, то чем тогда хороши массивы?

Массивы



На сайтах со всевозможными хит-парадами и «первыми десятками» применяется жульническая тактика для увеличения количества просмотров. Вместо того чтобы вывести весь список на одной странице, они размещают по одному элементу на странице и заставляют вас нажимать кнопку Next для перехода к следующему элементу. Например, «Десятка лучших злодеев в сериалах» не выводится на одной странице. Вместо этого вы начинаете с № 10 (Ньюман из «Сайнфелда») и нажимаете Next на каждой странице, пока не доберетесь до № 1 (Густаво Фринг из «Во все тяжкие»). В результате сайту удается показать вам рекламу на целых 10 страницах, но нажимать Next 9 раз для перехода к первому месту скучно. Было бы гораздо лучше, если бы весь список помещался на одной странице, а вы могли бы просто щелкнуть на имени человека для получения дополнительной информации.

Похожая проблема существует и у связанных списков. Допустим, вы хотите получить последний элемент связанного списка. Просто прочитать нужное значение не удастся, потому что вы не знаете, по какому адресу оно хранится. Вместо этого придется сначала обратиться к элементу № 1 и узнать адрес элемента № 2, потом обратиться к элементу № 2 и узнать адрес элемента № 3... и так далее, пока не доберетесь до последнего элемента. Связанные списки отлично подходят в тех ситуациях, когда данные должны читаться

последовательно: сначала вы читаете один элемент, по адресу переходите к следующему элементу и т. д. Но если вы намерены прыгать по списку туда-сюда, держитесь подальше от связанных списков.

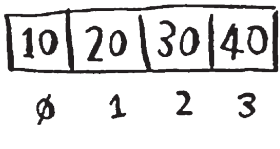
С массивами дело обстоит совершенно иначе. Работая с массивом, вы заранее знаете адрес каждого его элемента. Допустим, массив содержит пять элементов и вы знаете, что он начинается с адреса 00. По какому адресу хранится пятый элемент?



Простейшая математика дает ответ: это адрес 04. Массивы прекрасно подходят для чтения элементов в произвольных позициях, потому что обращение к любому элементу в массиве происходит мгновенно. В связанном списке элементы не хранятся рядом друг с другом, поэтому мгновенно определить позицию *i*-го элемента в памяти невозможно — нужно обратиться к первому элементу, чтобы получить адрес второго элемента, затем обратиться ко второму элементу для получения адреса третьего — и так далее, пока вы не доберетесь до *i*-го.

Терминология

Элементы массива пронумерованы, причем нумерация начинается с 0, а не с 1. Например, в этом массиве значение 20 находится в позиции 1.



А значение 10 находится в позиции 0. Неопытных программистов этот факт обычно вводит в ступор. Тем не менее выбор нулевой начальной позиции

упрощает написание кода по работе с массивами, поэтому программисты остановились на этом варианте. Почти во всех языках программирования нумерация элементов массива начинается с 0. Вскоре вы к этому привыкнете.

Позиция элемента называется его *индексом*. Таким образом, вместо того чтобы говорить «Значение 20 находится в позиции 1», правильно сказать «Значение 20 имеет индекс 1». В этой книге термин «индекс» означает то же, что и «позиция».

Ниже приведены примеры времени выполнения основных операций с массивами и списками.



Вопрос: почему вставка элемента в массив требует времени $O(n)$? Предположим, вы хотите вставить элемент в начало массива. Как бы вы это сделали? Сколько времени на это потребуется? Ответы на эти вопросы вы найдете в следующем разделе!

УПРАЖНЕНИЕ

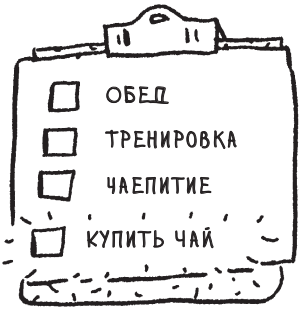
2.1 Допустим, вы строите приложение для управления финансами.

1. ПРОДУКТЫ
2. КИНО
3. ВЕЛОСИПЕДНЫЙ КЛУБ

Ежедневно вы записываете все свои траты. В конце месяца вы анализируете расходы и вычисляете, сколько денег было потрачено. При работе с данными выполняется множество операций вставки и относительно немного операций чтения. Какую структуру использовать — массив или список?

Вставка в середину списка

Предположим, вы решили, что список задач должен больше напоминать календарь. Прежде данные добавлялись только в конец списка, а теперь они должны добавляться в порядке их выполнения.

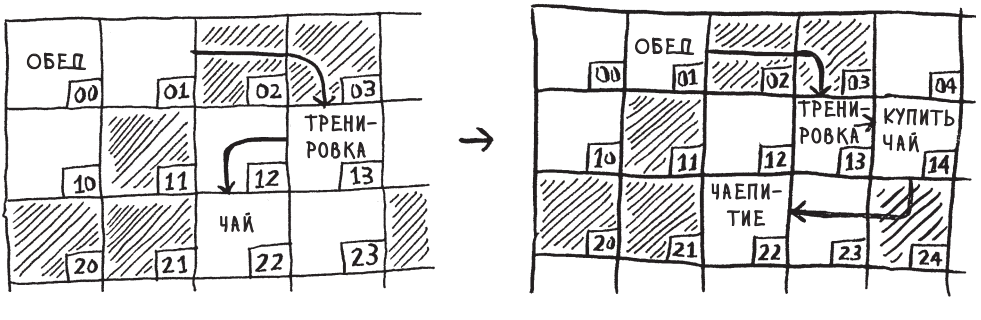


Неупорядоченный

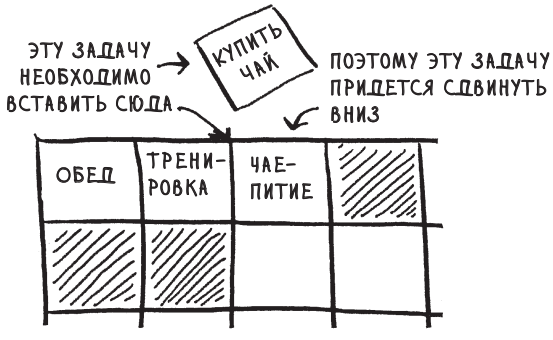


Упорядоченный

Что лучше подойдет для вставки элементов в середину: массивы или списки? Со списком задача решается изменением указателя в предыдущем элементе.



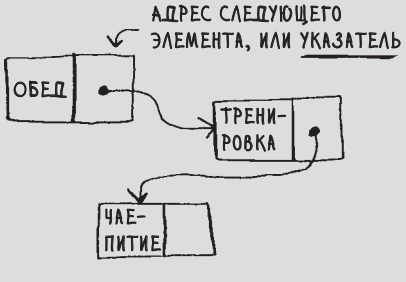
А при работе с массивом придется сдвигать все остальные элементы.



А если свободного места не осталось, все данные придется скопировать в новую область памяти! В общем, списки лучше подходят для вставки элементов в середину.

УКАЗАТЕЛИ

Я часто говорю о том, что каждый элемент связанного списка указывает на следующий элемент списка. Но как именно он это делает? С помощью указателей.



В каждом элементе связанного списка выделяется некоторый объем памяти для хранения адреса следующего элемента. Эта часть элемента называется указателем.

Периодически вы будете встречать этот термин, особенно если вы пишете на низкоуровневом языке, таком как C. Поэтому полезно знать, что он означает.

Удаление

Что, если вы захотите удалить элемент? И снова список лучше подходит для этой операции, потому что в нем достаточно изменить указатель в предыдущем элементе. В массиве при удалении элемента все последующие элементы нужно будет сдвинуть вверх.

В отличие от вставки, удаление возможно всегда. Попытка вставки может быть неудачной, если в памяти не осталось свободного места. С удалением подобных проблем не бывает.

Ниже приведены примеры времени выполнения основных операций с массивами и связанными списками.

	МАССИВЫ	СПИСКИ
ЧТЕНИЕ	O(1)	O(n)
ВСТАВКА	O(n)	O(1)
УДАЛЕНИЕ	O(n)	O(1)

Заметим, что вставка и удаление выполняются за время $O(1)$ только в том случае, если вы можете мгновенно получить доступ к удаляемому элементу. На практике обычно сохраняются ссылки на первый и последний элементы связанного списка, поэтому время удаления этих элементов составит всего $O(1)$.

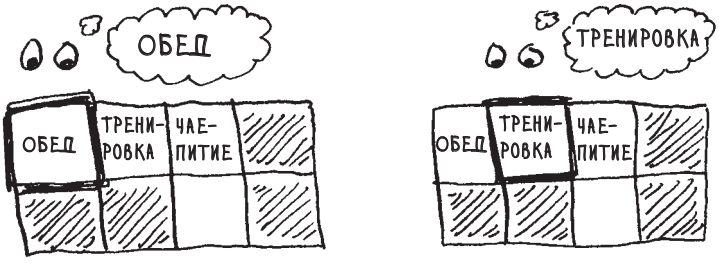
Какая структура данных используется чаще: массивы или списки?

Массивы используются часто, поскольку имеют множество преимуществ перед связанными списками. Во-первых, их проще читать. Во-вторых, массивы подерживают произвольный доступ.

Всего существуют два вида доступа: произвольный и последовательный. При последовательном доступе элементы читаются по одному, начиная с первого. Связанные списки поддерживают только последовательный

доступ. Если вы захотите прочитать 10-й элемент связанного списка, вам придется прочитать первые 9 элементов и перейти по ссылкам к 10-му элементу. Многие реальные ситуации требуют произвольного доступа, поэтому массивы часто применяются на практике.

Однако даже безотносительно произвольного доступа массивы работают быстрее, поскольку могут использовать кэширование. Возможно, вы представляете себе процесс чтения как восприятие одного элемента за раз.



Но на самом деле компьютеры читают целыми разделами, поскольку это значительно ускоряет переход к следующему элементу.

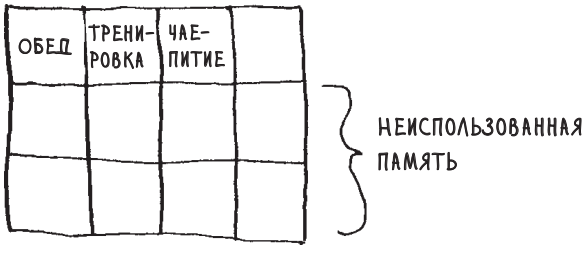


С массивами можно сделать то, что не получится со связанными списками. С помощью массива можно прочитать целый раздел элементов. В связанном списке местоположение следующего элемента неизвестно. Придется прочитать элемент, узнать, где находится следующий, а затем прочитать и его.

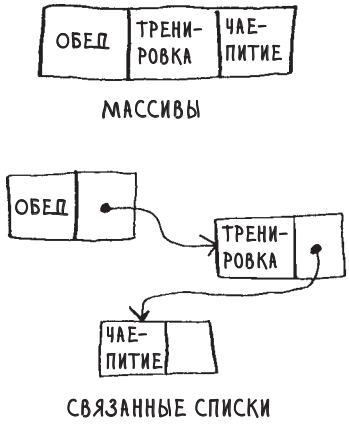
Таким образом, массивы обеспечивают не только произвольный, но и более быстрый последовательный доступ!

Массивы проще читать. А как насчет эффективного использования памяти? Помните, я говорил, что в случае с массивами обычно выделяют больше

памяти, чем нужно, и если в конечном итоге она не задействуется, то ее ресурс расходуется впустую?



На самом деле объем такой неиспользованной памяти получается не очень большим. С другой стороны, когда вы применяете связанный список, вы выделяете дополнительную память для каждого элемента, поскольку в ней сохраняется адрес следующего элемента. Таким образом, связанные списки занимают больше места, если их элементы относительно невелики. На рисунке показано, сколько места занимает одинаковый объем информации в массиве и в связанном списке. Видно, что связанный список гораздо больше.



Конечно, если элементы массива крупные, то даже один слот неиспользованной памяти может оказаться очень большим, а объем дополнительной памяти, используемой для хранения указателей, по сравнению с ним будет незначительным.

Таким образом, массивы используются чаще, чем связанные списки, за исключением особых случаев.

УПРАЖНЕНИЯ

2.2 Допустим, вы пишете приложение для приема заказов от посетителей ресторана. Приложение должно хранить список заказов. Официанты добавляют заказы в список, а повара читают заказы из списка и выполняют их. Заказы образуют очередь: официанты добавляют заказы в конец очереди, а повар берет первый заказ из очереди и начинает готовить.

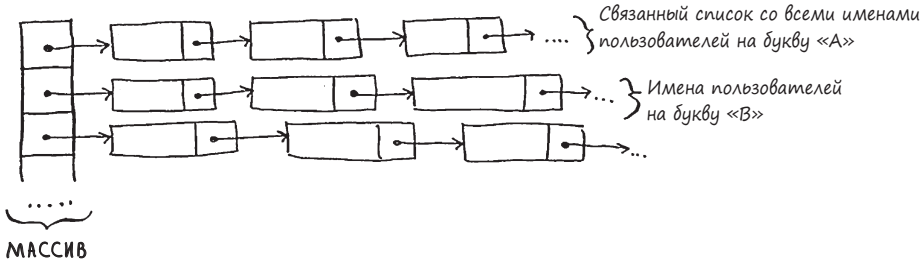


Какую структуру данных вы использовали бы для реализации этой очереди: массив или связанный список? (Подсказка: связанные списки хорошо подходят для вставки/удаления, а массивы — для произвольного доступа к элементам. Что из этого понадобится в данном случае?)

2.3 Проведем мысленный эксперимент. Допустим, Facebook хранит список имен пользователей. Когда кто-то пытается зайти на сайт Facebook, система пытается найти имя пользователя. Если имя входит в список имен зарегистрированных пользователей, то вход разрешается. Пользователи приходят на Facebook достаточно часто, поэтому поиск по списку имен пользователей будет выполняться часто. Будем считать, что Facebook использует бинарный поиск для поиска в списке. Бинарному поиску необходим произвольный доступ — алгоритм должен мгновенно обратиться к среднему элементу текущей части списка. Зная это обстоятельство, как бы вы реализовали список пользователей: в виде массива или в виде связанного списка?

2.4 Пользователи также довольно часто создают новые учетные записи на Facebook. Предположим, вы решили использовать массив для хранения списка пользователей. Какими недостатками обладает массив для выполнения вставки? Допустим, вы используете бинарный поиск для нахождения учетных данных. Что произойдет при добавлении новых пользователей в массив?

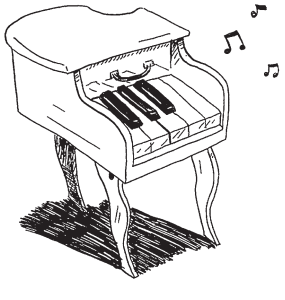
2.5 В действительности Facebook не использует ни массив, ни связанный список для хранения информации о пользователях. Рассмотрим гибридную структуру данных: массив связанных списков. Имеется массив из 26 элементов. Каждый элемент содержит ссылку на связанный список. Например, первый элемент массива указывает на связанный список всех имен пользователей, начинающихся на букву «А». Второй элемент указывает на связанный список всех имен пользователей, начинающихся на букву «В», и т. д.



Предположим, пользователь с именем «Adit B» регистрируется на Facebook и вы хотите добавить его в список. Вы обращаетесь к элементу 1 массива, находите связанный список элемента 1 и добавляете «Adit B» в конец списка. Теперь предположим, что зарегистрировать нужно пользователя «Zakhir H». Вы обращаетесь к элементу 26, который содержит связанный список всех имен, начинающихся с «Z», и проверяете, присутствует ли «Zakhir H» в этом списке.

Теперь сравните эту гибридную структуру данных с массивами и связанными списками. Будет ли она быстрее или медленнее каждой исходной структуры при поиске и вставке? Приводить «О-большое» не нужно, просто выберите одно из двух: быстрее или медленнее.

Сортировка выбором

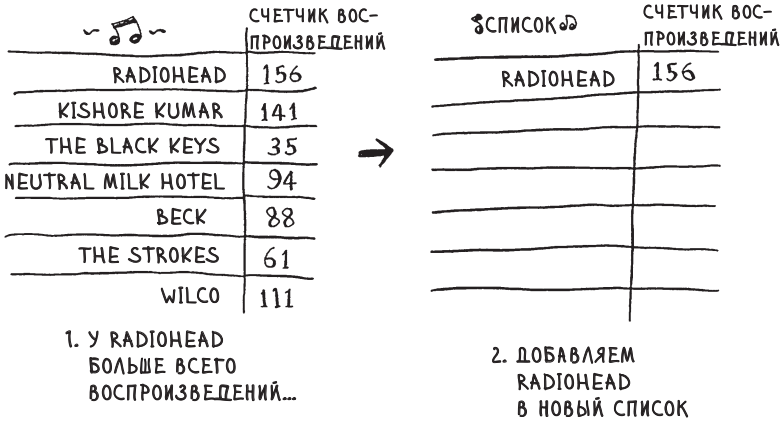


А теперь объединим все, что вы узнали, во втором алгоритме: сортировке выбором. Чтобы освоить этот алгоритм, вы должны понимать, как работают массивы и списки и «О-большое». Допустим, у вас на компьютере записана музыка и для каждого исполнителя хранится счетчик воспроизведений.

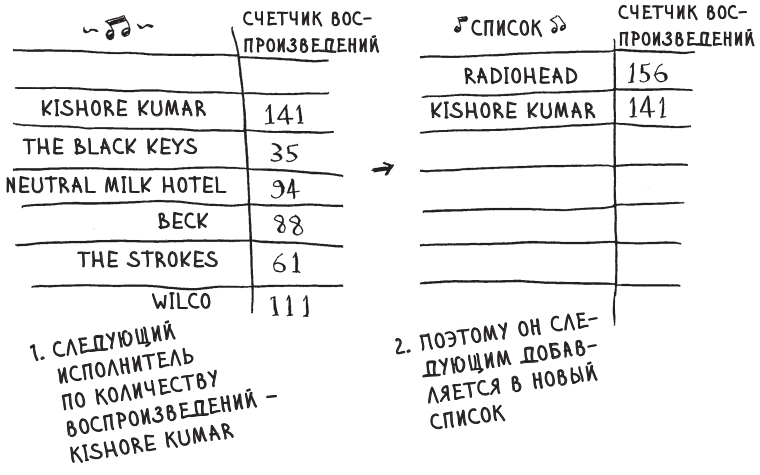
	Счетчик воспроизведений
RADIOHEAD	156
KISHORE KUMAR	141
THE BLACK KEYS	35
NEUTRAL MILK HOTEL	94
BECK	88
THE STROKES	61
WILCO	111

Вы хотите отсортировать список по убыванию счетчика воспроизведений, чтобы самые любимые исполнители стояли на первых местах. Как это сделать?

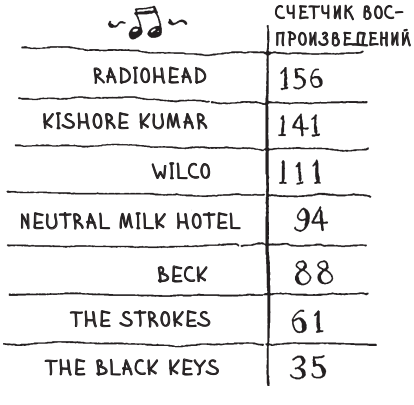
Одно из возможных решений — пройти по списку и найти исполнителя с наибольшим количеством воспроизведений. Этот исполнитель добавляется в новый список.



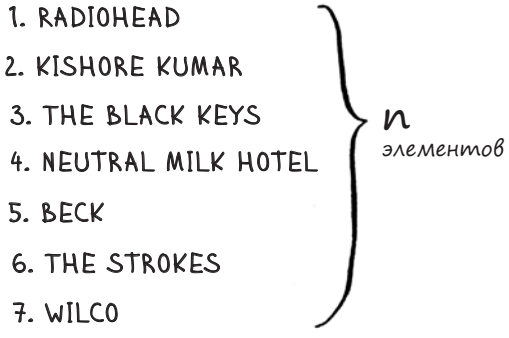
Потом то же самое происходит со следующим по количеству воспроизведений исполнителем.



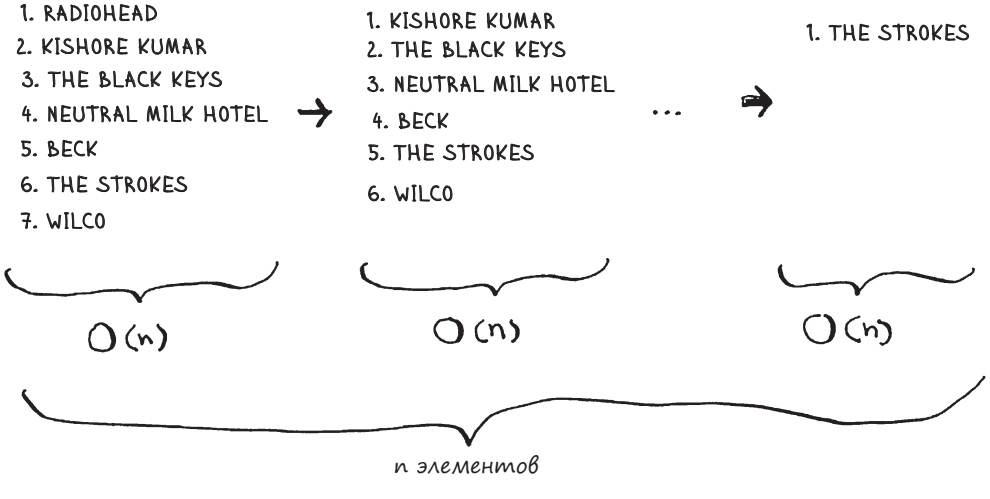
Продолжая действовать так, мы получаем отсортированный список.



А теперь попробуем оценить происходящее с точки зрения теории вычислений и посмотрим, сколько времени будут занимать операции. Напомним, что время $O(n)$ означает, что вы по одному разу обращаетесь к каждому элементу списка. Например, при простом поиске по списку исполнителей каждый исполнитель будет проверен один раз.



Чтобы найти исполнителя с наибольшим значением счетчика воспроизведения, необходимо проверить каждый элемент в списке. Как вы уже видели, это делается за время $O(n)$. Итак, имеется операция, выполняемая за время $O(n)$, и ее необходимо выполнить n раз:



Все это требует времени $O(n \times n)$, или $O(n^2)$.

Алгоритмы сортировки очень полезны. Например, вы можете отсортировать:

- имена в телефонной книге;
- даты путешествий;
- сообщения электронной почты (от новых к старым).

УМЕНЬШЕНИЕ КОЛИЧЕСТВА ПРОВЕРЯЕМЫХ ЭЛЕМЕНТОВ

При каждом выполнении операций количество элементов, которые нужно проверить, сокращается. Со временем все сведется к проверке всего одного элемента. Почему же время выполнения все равно оценивается как $O(n^2)$? Это хороший вопрос, и ответ на него связан с ролью констант в «О-большом». Тема будет более подробно рассмотрена в главе 4, но я кратко объясню суть. Вы правы, вам действительно не нужно каждый раз проверять весь список из n элементов. Сначала проверяются n элементов, потом $n - 1$, $n - 2$ ... 2, 1. В среднем проверяется список из $\frac{1}{2} \times n$ элементов. Его время выполнения составит $O(n \times \frac{1}{2} \times n)$. Однако константы (такие, как $\frac{1}{2}$) в «О-большом» игнорируются (еще раз: за полным обсуждением обращайтесь к главе 4), поэтому мы просто используем $O(n \times n)$, или $O(n^2)$.

Алгоритм сортировки выбором легко объясняется, но медленно работает. Быстрая сортировка — эффективный алгоритм сортировки, который выполняется за время $O(n \log n)$. Но мы займемся этой темой в главе 4!

Пример кода

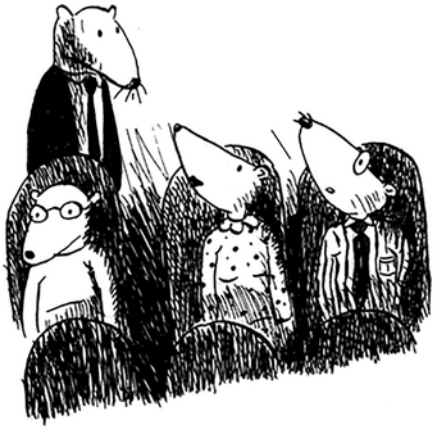
Мы не будем приводить код сортировки музыкального списка, но написанный ниже код делает нечто очень похожее: он выполняет сортировку массива по возрастанию. Напишем функцию для поиска наименьшего элемента массива:

```
def findSmallest(arr):
  smallest = arr[0]        <···· Для хранения наименьшего значения
  smallest_index = 0       <···· Для хранения индекса наименьшего значения
  for i in range(1, len(arr)):
    if arr[i] < smallest:
      smallest = arr[i]
      smallest_index = i
  return smallest_index
```

Теперь на основе этой функции можно написать функцию сортировки выбором:

```
def selectionSort(arr):                    Сортирует массив
  newArr = []
  for i in range(len(arr)):
      smallest = findSmallest(arr)         Находит наименьший элемент в массиве
      newArr.append(arr.pop(smallest))     и добавляет его в новый массив
  return newArr

print selectionSort([5, 3, 6, 2, 10])
```



Шпаргалка

- Память компьютера напоминает огромный шкаф с ящиками.
- Если вам потребуется сохранить набор элементов, воспользуйтесь массивом или списком.
- В массиве все элементы хранятся в памяти рядом друг с другом.
- В связанном списке элементы распределяются в произвольных местах памяти, при этом в каждом элементе хранится адрес следующего элемента.
- Массивы обеспечивают быстрое чтение.
- Списки обеспечивают быструю вставку и удаление.